

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Организация ЭВМ и систем»
Тема: Написание собственного прерывания и работа со строками

Студентка гр. 2304

Иванова М.А.

Преподаватель

Фирсов М. А.

Санкт-Петербург

2023

Цель работы.

Изучить прерывания и работу со строками и отработать на практике, разработав программу собственного обработчика и модифицирования строк на языке Ассемблер.

Задание. Вариант 10, 10и

1. Реализовать сценарий работы с прерываниями в соответствии с вариантом.

Замечания:

- 1) В сценариях опущены:

- Вывод приветственного сообщения и ввод строки пользователем. Это 1-ый пункт основной программы каждого сценария (поэтому основная программа во всех сценариях начинается с пункта 2). Слова «Ожидание ввода строки», которые встречаются в таблице сценариев, – это всего лишь организация задержки с ожиданием нажатия клавиш, для ввода данных не используется.

- Действия по восстановлению изменённых прерываний, если они не требуются по сценарию специально. После завершения программы все изменённые прерывания всегда должны быть восстановлены (независимо от сценария).

- 2) Для 1ch описан не алгоритм работы одного вызова, а последовательность действий, выполняемая в результате множества вызовов. Время должно измеряться через подсчёт количества выполненных вызовов, функцию задержки использовать не следует.

- 3) «вып. работу» означает, что надо на основе введённой строки (п. 1 основной программы) создать модифицированную строку и вывести её на экран. При выполнении преобразования нельзя портить исходную строку, результат преобразования должен записываться в выходную строку.

- 4) Перед запуском ожидания нажатия клавиши («ввод строки» в таблице) вывести сообщение об этом.

5) Для исключения возможного взаимного влияния системных и пользовательских прерываний рекомендуется отвести в программе под стек не менее 1Кбайт.

2. Реализовать преобразования строки с использованием команд работы со строками в соответствии с вариантом.

Замечания:

1) При выполнении преобразования обязательно использовать команды работы со строками

2) В любой системе счисления (СС) инверсная цифра определяется как разность: основание СС – 1 – исходная цифра. В шестнадцатеричной СС: исх. цифра = В(11), её инверсия: $16 - 1 - 11 = 4$.

3) Завершающие символы (0ah, 0dh, «\$») рассматривать именно как завершающие, т.е. не подвергать преобразованиям; закончить ими выходную строку.

Вариант 10, 10и:

№ сц.	Действия основной программы	Действия 1ch	Действия 9h
10	2) замена 1ch; 3) ожидание ввода строки.	после замены через 4 сек. заменяет 9h и восст. себя.	1) вып. работу; 2) восст. себя.

и) Инвертирование введённых во входной строке цифр в шестнадцатеричной СС.

Основные теоретические положения.

Прерывание – это процесс вызова процедур для выполнения некоторой задачи, обычно связанной с обслуживанием некоторых устройств (обработка сигнала таймера, нажатия клавиши и т.д.). Когда возникает прерывание, процессор прекращает выполнение текущей программы (если её приоритет ниже) и запоминает в стеке вместе с регистром флагов адрес возврата (CS:IP) – места, с которого будет продолжена прерванная программа. Затем в CS:IP

загружается адрес программы обработки прерывания и ей передаётся управление.

Адреса 256 программ обработки прерываний, так называемые векторы прерывания, имеют длину по 4 байта (в первых двух хранится значение IP, во вторых – CS) и хранятся в младших 1024 байтах памяти.

Программа обработки прерывания должна заканчиваться инструкцией IRET (возврат из прерывания), по которой из стека восстанавливается адрес возврата и регистр флагов.

Программа, использующая новые программы обработки прерываний, при своём завершении должна восстанавливать оригинальные векторы прерываний. Функция 35 прерывания 21H возвращает текущее значение вектора прерывания, помещая значение сегмента в ES, а смещение в BX.

Для задания адреса собственного прерывания с заданным номером в таблицу векторов прерываний используется функция 25H прерывания 21H, которая устанавливает вектор прерывания на указанный адрес нового обработчика.

При использовании функции 25H прерывания 21H DOS знает, что вы делаете, и гарантирует, что в момент записи прерывания будут заблокированы. Поэтому вызывать команды CLI и STI не нужно.

Прерывания бывают аппаратные (вызываемые в результате сигналов от оборудования) и программные (вызываемые в коде).

- 1CH и 08H – можно рассматривать их как аппаратные прерывания, генерируемые системным таймером; генерируются автоматически 18,2 раза в сек.;

- 09H – аппаратное прерывание, возникающее в результате нажатия клавиш клавиатуры;

- 21H – программное прерывание для вызова сервисов DOS.

Для обработки строковых данных ассемблер имеет пять групп команд обработки строк, среди них есть:

— LODS — загрузить из памяти один байт в регистр AL или одно слово в регистр AX;

— STOS — записать содержимое регистра AL или AX в память;

Каждая команда имеет модификации, указывающие размер операнда: байт (B), слово (W), двойное слово (D). Например: MOVSB, MOVSW, MOVSD.

Эти команды предполагают, что регистры DI и SI содержат относительные адреса, указывающие на необходимые области памяти (для загрузки можно использовать команду LEA). Регистр SI обычно связан с регистром сегмента данных – DS:SI. Регистр DI всегда связан с регистром дополнительного сегмента – ES:DI. Следовательно, команды MOVSB, STOS, CMPS и SCAS требуют инициализации регистра ES (обычно адресом в регистре DS).

Выполнение работы.

1. Для реализации работы с прерываниями в соответствии с вариантом, была написаны несколько процедур:
 - TimerInt - новый обработчик прерывания 1Ch;
 - KeyInt - новый обработчик прерывания 09h;
 - SaveTimerInt, SaveKeyInt – сохраняют значения векторов прерываний до их переопределения;
 - SetTimerInt, SetKeyInt – переопределяют значения векторов прерываний;
 - RestoreTimerInt, RestoreKeyInt – восстанавливают исходные значения векторов прерываний.
2. Для реализации работы со строками в соответствии с вариантом, была написаны процедуры:
 - StringChange – преобразует строки;
 - StringRead – читает строку из консоли;
 - StringWrite – выводит строку в консоль.

3. Программа протестирована, оформлена таблицы с входными данными и результатами (табл.1 и табл.2)

Таблица 1 – Результаты тестирования сценария работы с прерываниями

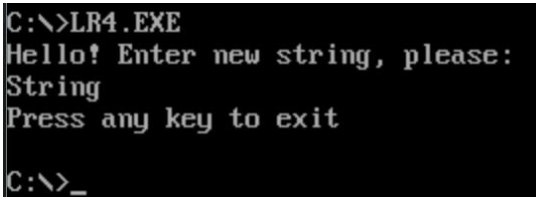
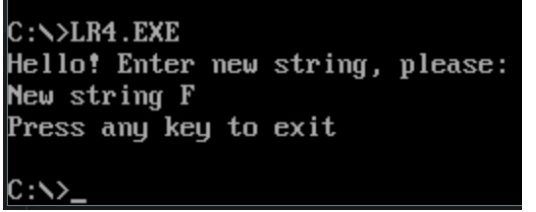
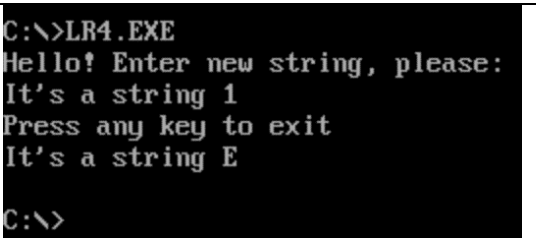
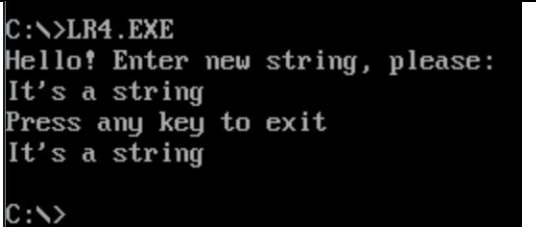
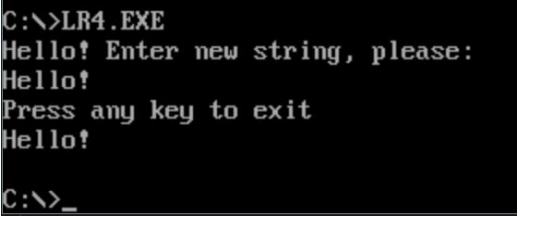
№	Прерывание	Комментарий	Скриншот
1	1Ch	Клавиша была нажата сразу после появления сообщения об ожидании ввода символа	
2	1Ch	Клавиша была нажата через 2 секунды после появления сообщения об ожидании ввода символа	
3	1Ch 09h	Клавиша была нажата через 4 секунды после появления сообщения об ожидании ввода символа	
4	1Ch 09h	Клавиша была нажата через 10 секунд после появления сообщения об ожидании ввода символа	

Таблица 2 – Результаты тестирования сценария преобразования строки

№	Входные данные	Выходные данные	Скриншот
1	Hello!	Hello!	

2	One is 1	One is E	<pre> C:\>LR4.EXE Hello! Enter new string, please: One is 1 Press any key to exit One is E C:\> </pre>
3	AOAOAOAOAOAO	5O5O5O5O5O5O	<pre> C:\>LR4.EXE Hello! Enter new string, please: AOAOAOAOAOAO Press any key to exit 5O5O5O5O5O5O C:\> </pre>
4	0123456789ABCDEF	FEDCBA9876543210	<pre> C:\>LR4.EXE Hello! Enter new string, please: 0123456789ABCDEF Press any key to exit FEDCBA9876543210 C:\>_ </pre>

Выводы.

Прерывания и работа со строками изучены и отработаны на практике. Разработана программу собственного обработчика и модифицирования строк на языке Ассемблер.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: LR4.ASM

```

DOSSEG
.MODEL    SMALL
.STACK    1024
.DATA

LF          EQU 0Ah          ; перенос строки (0Ah = 10)
CR          EQU 0Dh          ; возврат каретки (0Dh = 13)
EOF         EQU '$'          ; конец строки ('$' = 24h =
36)

INT_KEY     EQU 09h          ; аппаратное прерывание,
возникающее в результате нажатия клавиш клавиатуры
INT_WAIT_KEY EQU 16h          ; программное прерывание для
ожидания ввода символа с клавиатуры
INT_TIMER   EQU 1Ch          ; аппаратное прерывание,
автоматически генерируемое системным таймером 18,2 раз/сек
INT_DOS     EQU 21h          ; программное прерывание для
запроса функций DOS

READ_KEY     EQU 00h          ; чтение введенного символа
STR_WRITE    EQU 09h          ; вывод строки символов на
экран
STR_READ     EQU 0Ah          ; ввод с клавиатуры с
буферизацией
SET_INT      EQU 25h          ; установка вектора
прерывания на указанный адрес
GET_INT      EQU 35h          ; получение вектора
прерывания
; значение сегмента в
ES, смещение в BX

STR_MAX_LEN  EQU 50h          ; max количество символов в
строке (50h = 80)

STR_INPUT_HEAD DB STR_MAX_LEN,0 ;
заголовок строки
STR_INPUT     DB STR_MAX_LEN DUP('*'),CR,LF,EOF ;
введенная пользователем строка
STR_RES       DB STR_MAX_LEN DUP('*'),CR,LF,EOF ;
преобразованная строка
STR_GREETING  DB 'Hello! Enter new string, please:',CR,LF,EOF ;
строка-приветствие
STR_WAIT      DB 'Press any key to exit',CR,LF,EOF ;
строка-ожидание
STR_NEWLINE   DB CR,LF,EOF
; перенос на новую строку

KEEP_TIMER_IP DW 0           ; хранение смещения TIMER
KEEP_TIMER_CS DW 0           ; хранение сегмента прерывания
TIMER
KEEP_KEY_IP   DW 0           ; хранение смещения KEY

```



```

KEEP_KEY_CS      DW  0                      ; хранение сегмента прерывания
KEY

COUNTER          DW  49h                    ; счетчик для отсчета 4 сек. (49h
= 73)

.CODE
; чтение строки из консоли
StringRead PROC FAR
    push  ax                                ; помещает ax в стек
    push  bx                                ; помещает bx в стек

    mov   ah,STR_READ                      ; ah = STR_READ = 0Ah = функция
ввода строки
    push  dx                                ; помещает в стек dx = смещение
заголовка строки
    int   INT_DOS                          ; вызов обработчика прерываний
INT_DOS = 21h для функции ввода строки
    pop   bp                                ; bp = dx = смещение заголовка
строки
    xor    bx,bx                            ; очищение bx
    mov    bl,ds:[bp+1]                     ; bx = bl = количество введенных
символов
    add    bx,bp                            ; bx = bx + bp = адрес конечного
введенного символа
    add    bx,2                             ; bx = bx + 2 = байт, следующий
за финальным CR
    mov    word ptr [bx+1], 240Ah           ; добавить в конец LF = 0Ah и
'$' = 24h

    pop    bx                              ; возвращает bx из стека
    pop    ax                              ; возвращает ax из стека

    ret
StringRead ENDP

; вывод строки в консоль
StringWrite PROC FAR
    push  ax                                ; помещает ax в стек

    mov   ah,STR_WRITE                      ; ah = STR_WRITE = 09h =
функция вывода строки на экран
    int   INT_DOS                          ; вызов обработчика
прерываний INT_DOS = 21h для функции вывода строки на экран

    pop    ax                              ; возвращает ax из стека

    ret
StringWrite ENDP

; сохранение векторов прерывания
; перед вызовом прерывания INT_KEY = 09h
SaveKeyInt PROC FAR
    push  ax                                ; помещает ax в стек
    push  bx                                ; помещает bx в стек
    push  es                                ; помещает es в стек

```

```

        mov     ah,GET_INT                ; ah = GET_INT = 35h = функция
получения вектора прерывания
        mov     al,INT_KEY                ; al = INT_KEY = 09h =
                                           ; = аппаратное
прерывание, возникающее в результате нажатия клавиш клавиатуры
        int     INT_DOS                  ; вызов обработчика
прерываний INT_DOS = 21h для функции получения вектора
        mov     KEEP_KEY_IP,bx           ; KEEP_KEY_IP = bx = смещение
сегмента
        mov     KEEP_KEY_CS,es           ; KEEP_KEY_CS = es = значение
сегмента

        pop     es                       ; возвращает es из стека
        pop     bx                       ; возвращает bx из стека
        pop     ax                       ; возвращает ax из стека

        ret
SaveKeyInt ENDP

; восстановление векторов прерывания INT_KEY = 09h
RestoreKeyInt PROC FAR
        push    dx                       ; помещает dx в стек
        push    ax                       ; помещает ax в стек
        push    ds                       ; помещает ds в стек

        mov     dx,KEEP_KEY_IP           ; dx = KEEP_KEY_IP = смещение
сегмента
        mov     ax,KEEP_KEY_CS           ; dx = KEEP_KEY_CS = значение
сегмента
        mov     ds,ax                   ; ds = ax = KEEP_KEY_CS =
значение сегмента
        mov     ah,SET_INT               ; ah = SET_INT = 25h = функция
установки вектора прерывания
        mov     al,INT_KEY               ; al = INT_KEY = 09h =
                                           ; = аппаратное
прерывание, возникающее в результате нажатия клавиш клавиатуры
        int     INT_DOS                  ; вызов обработчика
прерываний INT_DOS = 21h для функции восстановления вектора

        pop     ds                       ; возвращает ds из стека
        pop     ax                       ; возвращает ax из стека
        pop     dx                       ; возвращает dx из стека

        ret
RestoreKeyInt ENDP

; преобразование введенной пользователем строки
StringChange PROC FAR
        push    ax                       ; помещает ax в стек
        push    bx                       ; помещает bx в стек
        push    cx                       ; помещает cx в стек
        push    si                       ; помещает si в стек
        push    di                       ; помещает di в стек
        push    dx                       ; помещает dx в стек

        mov     cx,STR_MAX_LEN           ; cx = STR_MAX_LEN = 80 =
максимальная длина строки
        add     cx,3                     ; cx = cx + 3

```

```

        lea    si,STR_INPUT                ; si = смещение STR_INPUT
= введенной пользователем строка
        lea    di,STR_RES                  ; di = смещение STR_RES =
преобразованной строки

        cld                                ; сброс флага df для
направления слева направо
StringChange_loop:                        ; инвертирование числа в
16 CC
        xor    ax,ax                      ; очищение ax
        lodsb                               ; загрузить из памяти один
байт из si в al = символ введенной пользователем строки
        cmp    al,'0'                    ; сравнивает al и '0'
        jl     StringChange_end           ; если al < '0', переходит к
StringChange_end
        cmp    al,'9'                    ; сравнивает al и '9'
        jg     StringChange_letter        ; если al > '9', переходит к
StringChange_letter
                                                ; если '0' <= al <=
'9', инвертирует число в 16 CC
        neg    ax                        ; меняет знак ax на
противоположный
        add    ax,6Fh                    ; ax = ax + 6Fh = ax + 111
= адрес инвертированной в цифру цифры в таблице ASCII
        cmp    al,'9'                    ; сравнивает al и '9'
        jle    StringChange_end           ; если al <= '9', переходит к
StringChange_end
        add    ax,7                      ; если al > '9', ax = ax +
7 = адрес инвертированной в букву цифры в таблице ASCII
        jmp    StringChange_end           ; переходит к StringChange_end
StringChange_letter:                      ; инвертирование числа,
записанного буквой, в 16 CC
        cmp    al,'A'                    ; сравнивает al и 'A'
        jl     StringChange_end           ; если al < 'A', переходит к
StringChange_end
        cmp    al,'F'                    ; сравнивает al и 'F'
        jg     StringChange_end           ; если al > 'F', переходит к
StringChange_end
                                                ; если 'A' <= al <=
'F', инвертирует число в 16 CC
        neg    ax                        ; меняет знак ax на
противоположный
        add    ax,76h                    ; ax = ax + 76h = ax + 118
= адрес инвертированной в цифру буквы в таблице ASCII
StringChange_end:                        ; запись символа в новую
строку
        stosb                               ; записать содержимое
регистра al = преобразованный символ
                                                ; в память по адресу
di = в преобразованную строку
        loop   StringChange_loop          ; цикл StringChange_loop,
cx - счетчик

        lea    dx,STR_RES                ; dx = смещение STR_RES =
преобразованной строки
        call   StringWrite                ; вызов процедуры StringWrite -
вывод строки на экран

```

```

        pop     dx                      ; возвращает dx из стека
        pop     ax                      ; возвращает ax из стека
        pop     bx                      ; возвращает bx из стека
        pop     cx                      ; возвращает cx из стека
        pop     si                      ; возвращает si из стека
        pop     di                      ; возвращает di из стека

        ret
StringChange ENDP

; обработчик прерывания INT_KEY = 09h
KeyInt PROC FAR

        call    StringChange           ; вызов процедуры
StringChange - преобразования введенной строки
        call    RestoreKeyInt          ; вызов процедуры
RestoreKeyInt - переопределения векторов прерывания

        push    ax                     ; помещает ax в стек
        mov     al, 20H                 ; al = 20H = команда конца
прерывания (EOI)
        out     20H, al                 ; записывает данные из al в порт
ввода-вывода 20H = контроллер прерываний
        pop     ax                     ; возвращает ax из стека

        jmp     dword ptr KEEP_KEY_IP   ; вызов обработчика прерывания
по адресу в KEEP_KEY_IP и KEEP_KEY_CS
KeyInt ENDP

; переопределение векторов прерывания INT_KEY = 09h
SetKeyInt PROC FAR
        push    ds                     ; помещает ds в стек
        push    dx                     ; помещает dx в стек
        push    ax                     ; помещает ax в стек

        mov     dx, OFFSET KeyInt       ; dx = смещение KeyInt
        mov     ax, SEG KeyInt           ; ax = база KeyInt
        mov     ds, ax                  ; ds = ax = база KeyInt
        mov     ah, SET_INT             ; ah = SET_INT = 25h = функция
установки вектора прерывания
        mov     al, INT_KEY              ; al = INT_KEY = 09h =
; = аппаратное
прерывание, возникающее в результате нажатия клавиш клавиатуры
        int     INT_DOS                 ; вызов обработчика
прерываний INT_DOS = 21h для функции восстановления вектора

        pop     ax                     ; возвращает ax из стека
        pop     dx                     ; возвращает dx из стека
        pop     ds                     ; возвращает ds из стека

        ret
SetKeyInt ENDP

; сохранение векторов прерывания
; перед вызовом прерывания INT_TIMER = 1Ch
SaveTimerInt PROC FAR
        push    ax                     ; помещает ax в стек

```

```

    push    bx                      ; помещает bx в стек
    push    es                      ; помещает es в стек

    mov     ah,GET_INT              ; ah = GET_INT = 35h = функция
получения вектора прерывания
    mov     al,INT_TIMER            ; al = INT_TIMER = 1Ch =
аппаратное прерывание,
                                     ; автоматически
генерируемое системным таймером 18,2 раз/сек
    int     INT_DOS                 ; вызов обработчика
прерываний INT_DOS = 21h для функции получения вектора
    mov     KEEP_TIMER_IP,bx        ; KEEP_TIMER_IP = bx = смещение
сегмента
    mov     KEEP_TIMER_CS,es        ; KEEP_TIMER_CS = es = значение
сегмента

    pop     es                      ; возвращает es из стека
    pop     bx                      ; возвращает bx из стека
    pop     ax                      ; возвращает ax из стека

    ret
SaveTimerInt ENDP

; восстановление векторов прерывания INT_TIMER = 1Ch
RestoreTimerInt PROC FAR
    push    dx                      ; помещает dx в стек
    push    ax                      ; помещает ax в стек
    push    ds                      ; помещает ds в стек

    mov     dx,KEEP_TIMER_IP        ; dx = KEEP_TIMER_IP =
смещение сегмента
    mov     ax,KEEP_TIMER_CS        ; dx = KEEP_TIMER_CS = значение
сегмента
    mov     ds,ax                  ; ds = ax = KEEP_TIMER_CS
= значение сегмента
    mov     ah,SET_INT              ; ah = SET_INT = 25h = функция
установки вектора прерывания
    mov     al,INT_TIMER            ; al = INT_TIMER = 1Ch =
аппаратное прерывание,
                                     ; автоматически
генерируемое системным таймером 18,2 раз/сек
    int     INT_DOS                 ; вызов обработчика
прерываний INT_DOS = 21h для функции восстановления вектора

    pop     ds                      ; возвращает ds из стека
    pop     ax                      ; возвращает ax из стека
    pop     dx                      ; возвращает dx из стека

    ret
RestoreTimerInt ENDP

; обработчик процедуры INT_TIMER = 1Ch
TimerInt PROC FAR
    push    ax                      ; помещает ax в стек

    dec     COUNTER                 ; COUNTER = COUNTER - 1 =
72
    mov     ax,COUNTER              ; ax = COUNTER = 72

```

```

    test    ax,ax                ; ax & ax
    jnz     TimerInt_End        ; если ax & ax != 0 (ax !=
0), переходит к TimerInt_End

    call    RestoreTimerInt     ; вызов процедуры RestoreTimerInt
-
                                ; - восстановления
векторов прерывания INT_TIMER = 1Ch
    call    SetKeyInt           ; вызов процедуры SetKeyInt -
                                ; - установки новых
векторов прерывания INT_KEY = 09h

TimerInt_End:
    pop     ax                  ; возвращает ax из стека
    iret                      ; возвращает cs и ip из стека
TimerInt ENDP

; переопределение векторов прерывания INT_TIMER = 1Ch
SetTimerInt PROC FAR
    push    ds                  ; помещает ds в стек
    push    dx                  ; помещает dx в стек
    push    ax                  ; помещает ax в стек

    mov     dx,OFFSET TimerInt  ; dx = смещение TimerInt
    mov     ax,SEG TimerInt     ; ax = база TimerInt
    mov     ds,ax               ; ds = ax = база TimerInt
    mov     ah,SET_INT          ; ah = SET_INT = 25h = функция
установки вектора прерывания
    mov     al,INT_TIMER        ; al = INT_TIMER = 1Ch =
аппаратное прерывание,
                                ; автоматически
генерируемое системным таймером 18,2 раз/сек
    int     INT_DOS             ; вызов обработчика
прерываний INT_DOS = 21h для функции восстановления вектора

    pop     ax                  ; возвращает ax из стека
    pop     dx                  ; возвращает dx из стека
    pop     ds                  ; возвращает ds из стека

    ret
SetTimerInt ENDP

; головная процедура
Main PROC FAR
    push    ds                  ; помещает ds в стек
    sub     ax,ax               ; очищает ax
    push    ax                  ; помещает ax в стек

    mov     ax,@data            ; ax = адрес начала
сегмента данных
    mov     ds,ax               ; ds = ax = адрес начала
сегмента данных
    mov     es,ax               ; es = ax = адрес начала
сегмента данных

    lea     dx,STR_GREETING     ; dx = смещение STR_GREETING =
строки-приветствия

```

```

    call    StringWrite                ; вызов процедуры StringWrite -
вывода строки STR_GREETING на экран

    lea     dx,STR_INPUT_HEAD          ; dx = смещение
STR_INPUT_HEAD = ?????????????????????????????????????
    call    StringRead                ; вызов процедуры StringRead -
чтения строки в STR_INPUT

    lea     dx,STR_NEWLINE             ; dx = смещение STR_NEWLINE =
переноса строки
    call    StringWrite                ; вызов процедуры StringWrite -
вывода строки STR_NEWLINE на экран

    call    SaveTimerInt              ; вызов процедуры
SaveTimerInt -                               ; - сохранения
векторов прерывания перед INT_TIMER = 1Ch
    call    SaveKeyInt                ; вызов процедуры SaveKeyInt -
                                           ; - сохранения
векторов прерывания перед INT_KEY = 09h
    call    SetTimerInt               ; вызов процедуры SetTimerInt -
                                           ; - установки новых
векторов прерывания INT_TIMER = 1Ch

    lea     dx,STR_WAIT               ; dx = смещение STR_WAIT =
строки-ожидания
    call    StringWrite                ; вызов процедуры StringWrite -
вывода строки STR_WAIT на экран

    mov     ah,READ_KEY                ; ah = READ_KEY = 00h =
функция чтения введенного символа
    int     INT_WAIT_KEY              ; вызов обработчика
прерываний INT_WAIT_KEY = 16h
                                           ; для функции для
ожидания ввода символа с клавиатуры

    call    RestoreKeyInt              ; вызов процедуры
RestoreKeyInt -                               ; - восстановления
векторов прерывания INT_KEY = 09h
    call    RestoreTimerInt           ; вызов процедуры RestoreTimerInt
-                                           ; - восстановления
векторов прерывания INT_TIMER = 1Ch

    ret
Main      ENDP
END Main

```